Paso 19 — Walk Forward Matrix (WFM): qué se calcula, con qué estadística y qué criterio hay cargado

Fecha: 2026-10-07 · **Para:** el dueño, que va a fijar los criterios él mismo · **Alcance:** sólo el paso 19: la tarea «WFM» de SQX (`sqx/projects/wfm.py`, skill `.claude/skills/wfm/`) y su análisis en Python (`studies/optimisation/wfm/`, `core/wfmatrix.py`, `core/wftrades.py`).

Qué es: el documento de consulta del paso 19, con la misma forma que el del paso 16 (`criterios-paso-16-spp-2026-10-07`). Explica desde cero la estadística que este paso añade, repasa **todos** los números que escriben SQX y el estudio, cita el criterio tal como está hoy en `pipeline/autopilot/criteria.yaml` (ninguna regla, y además ningún hecho guardado), y cierra con lo que no cuadra y con las decisiones que te quedan.

De dónde salen los números. Todos los ejemplos son de la corrida `Trade_NIKKEI225_M30_volatilidadL` del 2026-10-04 (NIKKEI225, M30, largas): las mismas diez estrategias del dossier del paso 16, 30 celdas cada una, 3.300 tramos de walk-forward y 231.686 operaciones exportadas. Están **recalculados desde el export** (`AlgoData/raw/Trade_NIKKEI225_M30_volatilidadL/WFM/2026-10-04/wfm/`) con el código del estudio y comparados con lo que quedó en disco (`AlgoData/reports/…/WFM/2026-10-04/wfm/verdict.csv`): las diez ρ, sus intervalos y sus veredictos coinciden; las diez condiciones de las 300 celdas coinciden con las que guardó SQX; la regla del área reconstruida coincide con la marca de SQX en las diez. La configuración de la tarea se ha leído del `project.cfx` archivado de esa corrida (`AlgoData/projects/snapshots/…/20261004-210011/`). Lo que el estudio no calcula —las celdas efectivas, el reparto de `oos2`, las tablas de sensibilidad, las simulaciones sin ventaja— lo he calculado yo y lo digo en cada sitio. «Ilustrativo» quiere decir cuenta hecha a mano o simulación para enseñar, no medida.

Revisión. Revisado por dos miembros del Council (Feynman, el código; Taleb, la estadística) el 2026-10-07; correcciones aplicadas. Feynman reprodujo las condiciones, el score de las 300 celdas, los rectángulos y los ejemplos de juguete; Taleb rehízo la simulación sin ventaja con otra semilla y 400 historias.

Ninguna línea de este documento está congelada. `criteria.yaml:59` lo dice: provisional hasta que una población sin ventaja haya recorrido la cadena; los pasos 17, 18, 19, 21, 23 y 24 «anotan solamente: sin regla a propósito».

---

0. En una página

La pregunta del paso. Si en vez de fijar los parámetros una vez los hubiéramos vuelto a optimizar cada cierto tiempo, operando después con lo recién optimizado, ¿habría ganado la estrategia? ¿Y lo habría hecho con muchas maneras distintas de trocear la historia, o sólo con una?

Qué hace. SQX repite ese ejercicio 30 veces por estrategia —6 números de tramos (6, 8, 10, 12, 14, 16) × 5 tamaños de tramo fuera de muestra (20, 25, 30, 35, 40 %)— sobre una ventana que va del principio de `build` al final de `oos2`. A cada una de las 30 celdas le pasa diez condiciones; una celda «pasa» si cumple el 80 % de ellas; la estrategia «pasa» si hay un rectángulo de 4×4 celdas con al menos 12 que pasan. Después Python lee los tramos y contesta otra pregunta: dentro de cada celda, ¿los tramos en que la

optimización salió mejor fueron también los tramos en que después fue mejor? (ρ de Spearman, y un veredicto: predice / ciega / perversa). Y en una pestaña aparte mide la estrategia **original**, sin reoptimizar, sólo en `oos2`, contra el activo comprado y contra traders al azar.

Qué hay cargado hoy. Nada. `pipeline/autopilot/criteria.yaml` no tiene entrada `"19"`. La marca de SQX no borra (`DeleteFailedStrategies=false`): las diez entran y las diez salen. Además, a diferencia del paso 16, el paso 19 **no deja** `hechos.parquet`: el piloto no lo juzga (`pipeline/autopilot/tail.py:89-99`), así que una regla escrita hoy no tendría hechos que leer hasta que se cambie eso. Lo único que baja al paso siguiente es el veredicto de la ρ, que el paso 20 (`blindJoint`) cuenta como una de sus piezas: «perversa» es un fallo allí y marca DESCARTAR en el `verdict.csv` del paso 20. Pero el autopiloto **no aplica** esa marca (el paso 20 tampoco tiene criterio ni databank que cortar): todas siguen, y sólo se corta si se aplica a mano con `/curate`.

Qué sale en la corrida real.

	pasa el área de SQX	**veredicto ρ**	**contra el activo en `oos2`**	**contra el azar en `oos2`**
de 10	**3** (C, E, G)	1 predice (F) · 4 ciega · **5 perversa**	10 «no se distingue»	1 con p ≤ 0,05

Las dos lecturas no coinciden en nada: la única estrategia que «predice» (F) suspende el área de SQX con 8 celdas de 30, y la que mejor pasa el área (G, 27 celdas de 30) es «perversa».

Lo que conviene saber antes de decidir (el detalle está en la sección 5):

1. **Las 30 celdas no son 30 pruebas.** Sus curvas fuera de muestra se parecen entre sí (correlación mediana 0,40 a 0,65): equivalen a un puñado de celdas independientes —entre 1,6 y 4,6 según cómo se cuente (2.8)—. «12 de 16» suena a mayoría amplia y vale por dos o tres tiradas. El código no calcula este número.
2. **El «fuera de muestra» de la WFM sólo lo es para los parámetros.** Entre el 67 y el 79 % de los días que la WFM llama OOS caen en `build` y `oos1`, dos tramos que la estrategia ya vio (el generador la construyó en uno y el paso 8 la eligió con el otro). Sólo `oos2` —2,75 años— es limpio, y la tarea se lo cobra con el spread de `oos1` (474,91) en vez del suyo (796,12).
3. **La condición «score» cuenta el primer tramo de optimización como si fuera resultado.** Su numerador es «primera ventana optimizada + OOS cosido». Con sólo el OOS, esa condición la cumpliría el 34 % de las celdas en vez del 71 %, y E dejaría de pasar el área.
4. **El veredicto de la ρ no distingue nada, y «perversa» sale sola.** El problema principal es el intervalo: trata las 30 celdas como independientes y sale entre cuatro y cinco veces demasiado estrecho. En simulaciones de estrategias sin ninguna ventaja, con esta misma rejilla, el estudio dice «perversa» entre el 64 y el 74 % de las veces. Parte es un sesgo negativo de la ρ (demostrado: desaparece al quitar el solape), pero **aun quitando el sesgo** el veredicto sale «no ciega» el 70 % de las veces, repartido entre perversa y predice. El paso 20 lo cuenta como fallo y marca DESCARTAR, aunque el autopiloto no aplica esa marca.
5. **La ρ no mide lo que dice su título.** No compara configuraciones mejor y peor optimizadas: compara, a lo largo de 6 a 16 tramos de una celda, lo bien que le fue al ganador dentro con lo bien que le fue fuera. Los candidatos descartados no se guardan.

6. **El modo de la tarea ha cambiado en disco y no en los documentos.** `assets/_build.yaml:483` dice hoy `simulated_is_simulated_oos`; su propio comentario, la skill y la corrida real dicen `exact_is_exact_oos`. `git diff` confirma que es un cambio sin confirmar. Una corrida lanzada hoy no sería el estudio que describen los papeles.
7. **Los porcentajes de calibración de las diez condiciones son del oro.** Vienen de 150 celdas de XAUUSD; en NIKKEI225 dos de ellas se cumplen la mitad o el doble de veces.
8. **El «% OOS» no es un porcentaje de la pasada.** Es el tramo fuera de muestra como porcentaje de la ventana de optimización: «20 %» es el 16,7 % de cada pasada.
9. **Ninguna p de la pestaña del azar es utilizable todavía.** El beneficio que esa tabla atribuye a la estrategia (7.021,62 en C) no es el de SQX (4.337,65) y la diferencia está sin explicar (5.10). Hasta cerrarlo, las p de 3.9 son orientativas.

---

1. Qué es el paso 19 y cómo decide

1.1 Dónde está

Es el cuarto y último de los estudios de robustez de parámetros (16 SPP, 17 WFC, 18 CSCV, 19 WFM), todos sobre las supervivientes del paso 14. El paso 20 (`blindJoint`) lee los cuatro juntos. Entra el databank «SPP OOS» y sale el databank «WFM». En la corrida real: `19 | sqx | 1695 s | WFM: 0 → 10` — 28 minutos, 170 segundos por estrategia.

1.2 La tarea de SQX, tal como corrió

Leída del `project.cfx` archivado (tarea `Retest-Task2.xml`, título «WFM») y comparada con `assets/_build.yaml:447-565` y `sqx/projects/wfm.py`:

ajuste	valor en la corrida	dónde se escribe
ventana	2011-09-19 → 2026-09-30 (`build..oos2` , 15,03 años)	`wfm.py:127-134`
costes	los de `oos1` : spread 474,91, slippage 166,22	`wfm.py:128` , `_build.yaml` `costs_segment`
motor, precisión	MetaTrader 5 con cobertura, precisión 2	`_build.yaml` `precision`
tipo de walk-forward	**2 = IS exacto / OOS exacto**	`wfm.py:28-30` ; hoy el YAML dice otra cosa (5.1)
ventana de optimización	flotante (se desplaza; no crece)	`optimization="15"`
eje de columnas	% OOS: 20, 25, 30, 35, 40	`wfm.py:55-76`
eje de filas	tramos: 6, 8, 10, 12, 14, 16	ídem
rango de cada parámetro	SQX recibe ±35 % y un máximo de 18 escalones (derivado de 2 × 35 / 4); el 4 % no se escribe en la tarea	`wfm.py:67-72`
pruebas por optimización	como mucho 5.000	`wfm.py:95`
aceptación	celda: 80 % de las condiciones · área: 4×4 con ≥ 12	`wfm.py:98-100`
qué hace con la que no pasa	la marca; no la borra	`DeleteFailedStrategies=false`

Los tres tramos del activo dentro de esa ventana:

tramo	fechas	años	spread declarado
`build`	2011-09-19 → 2019-12-31	8,28	304,11
`oos1`	2020-01-01 → 2023-12-31	4,00	474,91
`oos2`	2024-01-01 → 2026-09-30	2,75	796,12

1.3 Qué exporta SQX y qué lee Python

`core/wfmatrix.py` y `core/wftrades.py` convierten el resultado de SQX en tablas. Para la corrida:

tabla	filas	qué es
`cells`	300	una por estrategia y celda: estadísticas del primer IS, del OOS cosido y de «todo»
`steps`	3.600	una por tramo: fechas y métricas de la optimización y de la pasada; 300 son «futuros» (1.4)
`params`	3.598	los parámetros elegidos en cada tramo (faltan 2: ver 5.13)
`conditions`	3.000	las diez condiciones de cada celda: valor y si se cumple
`objectives`	300	todos los objetivos que SQX calcula por celda, se usen o no
`trades`	231.686	operaciones de las 300 celdas: 186.019 fuera de muestra, 45.667 del primer IS
`main`	4.104	las operaciones de las diez estrategias **originales** en toda la ventana
`status`	10	la marca de SQX; 7 con «FAILED»

1.4 El tramo «futuro»

Cada celda tiene un tramo más que el número que la nombra: en la celda de 6 tramos hay 7 optimizaciones. La séptima optimiza sobre la última ventana (acaba el 2026-09-30) y su «pasada» sería del 2026-09-30 al 2028-02-11: todavía no existe. SQX la guarda porque de ahí salen los «parámetros recomendados». El estudio la descarta en todas sus cuentas (`config.yaml` `drop_future: true`; `inputs/export.py:24-44`), con una excepción que se ve en 3.6. Por estrategia: 360 tramos = 5 × (7+9+11+13+15+17), de los que 330 son reales.

1.5 Quién decide qué

quién	qué dice	qué consecuencia tiene hoy
SQX, regla del área	pasa / «FAILED»	ninguna: la estrategia sigue; el estudio la anota como aviso `failed_en_sqx`
el estudio, ρ	predice / ciega / perversa	es la pieza «19 wfm» del paso 20: predice = bien, ciega = vigilar, perversa = fallo → DESCARTAR en el `verdict.csv` del 20, que el autopiloto no aplica (4.2)
el estudio, pestaña «Contra el activo y el azar»	diferencia de Sharpe y p del azar	ninguna: sólo se enseña
`criteria.yaml`	nada	ninguna

1.6 Tres avisos

- **No hay Limbo posible hoy** porque no hay regla. El día que la haya: un hecho que falte es Limbo, no Pass. Con el estudio actual faltaría la ρ de una celda con menos de 5 tramos (`min_steps`), cosa que en esta rejilla no ocurre (el mínimo son 6), y el trader al azar de una estrategia con menos de 30 operaciones dentro de `oos2` (`monkey_min_trades`), que tampoco ocurre aquí (la que menos, 45).
- **Este paso toca `oos2`.** `assets/_policy.yaml` lo reserva para seis estudios, y la WFM es uno. Qué lee exactamente y qué gasta está en 2.9.
- **El trader al azar se lee sólo fuera de muestra.** La pestaña lo cumple: recorta a `oos2` antes de sortear (`inputs/asset.py:24`). En `build` no se sortea nada.

1.7 Las estrategias que nos acompañan

Las mismas cuatro letras del dossier del paso 16, y las diez con su nombre:

letra	estrategia	celdas que pasan	área	ρ	veredicto
	`10.11.484`	13	no (10)	−0,02	ciega
	`13.3.390`	2	no (2)	−0,13	perversa
C	`14.3.479`	13	**sí (12)**	−0,06	ciega
F	`15.9.352`	8	no (8)	**+0,35**	**predice**
	`16.5.352`	5	no (5)	+0,02	ciega
	`18.5.355`	9	no (7)	−0,19	perversa
	`19.2.363`	0	no (0)	−0,16	perversa
E	`3.11.426`	17	**sí (12)**	−0,02	ciega
	`3.9.284`	9	no (5)	−0,42	perversa
G	`4.13.338`	27	**sí (16)**	−0,25	perversa

«Área» lleva entre paréntesis el mejor rectángulo de 4×4: cuántas de sus 16 celdas pasan.

---

2. Fundamentos de este paso

2.0 Lo que ya sabes de los pasos anteriores, en seis líneas

- **Dentro de muestra (IS)** es el dato con el que se eligió algo; **fuera de muestra (OOS)** es dato que no intervino en esa elección. Lo que importa es *qué elección*: un dato puede ser OOS para una cosa e IS para otra.
- **Profit factor (PF):** lo ganado en las operaciones buenas dividido entre lo perdido en las malas. 1 es empate.
- **Sharpe:** la media de los resultados dividida entre lo que varían. Mide regularidad, no tamaño.
- **p-valor:** en qué fracción de los mundos sin ventaja saldría un resultado al menos tan bueno. Pequeño = difícil de explicar por suerte.
- **Intervalo del 95 %:** el rango en el que caería la medida 95 de cada 100 veces si se repitiera el experimento. Si contiene el cero, no se puede decir que el número sea distinto de cero.
- **Muchas pruebas:** si miras diez estrategias al 5 %, la probabilidad de que alguna «pase» por pura suerte es $1 - 0{,}95^{10} = 40\ \%$.

2.1 Qué es un walk-forward

Optimizar una estrategia sobre toda la historia y mirar cuánto gana ahí es hacerse trampas: los parámetros se eligieron viendo las respuestas. El walk-forward lo arregla imitando lo que harías en real:

1. coges un trozo de historia (la **ventana de optimización**, o IS) y buscas los mejores parámetros;
2. operas con ellos el trozo **siguiente** (la **pasada**, o tramo OOS), que la optimización no vio;
3. avanzas las dos ventanas un tramo y repites.

Ejemplo de juguete. Diez años de historia, ventana de optimización de 5 años, pasada de 1 año:

```
tramo   optimiza con los años    opera el año
  1          1 2 3 4 5               6
  2          2 3 4 5 6               7
  3          3 4 5 6 7               8
  4          4 5 6 7 8               9
  5          5 6 7 8 9              10
```

Tres cosas se ven ya en el dibujo:

- los años 1 a 5 no se operan nunca fuera de muestra: sólo sirven para la primera optimización;
- cada ventana de optimización comparte 4 de sus 5 años con la anterior: se solapan un 80 %;
- la ventana del tramo 2 contiene el año 6, que es justo el que se operó en el tramo 1. Esto vuelve en 2.7 y es importante.

SQX admite que la ventana crezca (anclada) o se desplace (flotante). La tarea usa la flotante, como el dibujo.

2.2 Cómo se cosen los tramos

Los cinco años operados (6, 7, 8, 9, 10) se ponen uno detrás de otro y forman una sola curva: el **OOS cosido**. Cada año lleva los parámetros que salieron de su propia optimización, así que la curva cambia de parámetros cuatro veces.

```
año            6      7      8      9     10    cosido
beneficio   +300   −100   +200    +50    −50     +400
```

Todas las métricas «OOS» de una celda (beneficio, PF, drawdown) se calculan sobre esa curva cosida. SQX guarda tres muestras de cada celda (`core/wfmatrix.py:17`):

muestra	qué contiene	en el ejemplo
IS (10)	sólo la **primera** ventana de optimización, con sus parámetros óptimos	años 1-5
OOS (20)	el cosido	años 6-10
todo (127)	la primera ventana + el cosido	años 1-10

La muestra «todo» es una curva rara: sus primeros cinco años son un resultado optimizado mirando esos mismos cinco años. Esto vuelve en 3.3.

2.3 Qué es una celda y qué es la matriz

El dibujo de 2.1 es **una** manera de trocear: 5 tramos, pasada de 1 año sobre ventana de 5. Hay muchas otras igual de razonables, y el resultado cambia con el troceo. La **matriz** repite el walk-forward con varias:

- **filas — tramos (R):** cuántas veces se reoptimiza: 6, 8, 10, 12, 14, 16;
- **columnas — % OOS (p):** el largo de la pasada como porcentaje del largo de la ventana de optimización: 20, 25, 30, 35, 40.

Una **celda** es un walk-forward entero con un par (R, p). Son 6 × 5 = 30 celdas, y cada una produce su propio OOS cosido y sus propias diez condiciones.

Cómo salen los largos. Si la historia entera dura T años, la ventana de optimización L y cada pasada p × L, las R pasadas más la primera ventana tienen que caber justas:

```
L + R × p × L = T        →        L = T / (1 + R × p)

Juguete: T = 10, R = 5, p = 20 %   →  L = 10 / (1 + 5 × 0,20) = 10 / 2 = 5 años; pasada = 1
año   ✓ (el dibujo de 2.1)
Real:    T = 15,03, R = 6, p = 20 % →  L = 15,03 / 2,2 = 6,83 años; pasada = 1,37 años
Real:    T = 15,03, R = 16, p = 40 % → L = 15,03 / 7,4 = 2,03 años; pasada = 0,81 años
```

Medido sobre las fechas del export (igual para las diez estrategias):

celda	ventana de optimización	cada pasada	OOS cosido	el OOS empieza
6 × 20	6,81 años	1,37	8,2	2018-07-14
6 × 40	4,42	1,77	10,6	2016-02-19
10 × 30	3,75	1,13	11,3	—
16 × 20	3,55	0,71	11,5	—
16 × 40	2,02	0,81	13,0	2013-09-26

Dos cosas que el nombre de los ejes no dice. **Una:** «20 % OOS» es 20 % de la ventana de optimización, no de la pasada completa (1,37 / (6,81 + 1,37) = 16,7 %). **Dos:** más tramos no quiere decir pasadas mucho más cortas; quiere decir sobre todo **ventanas de optimización más cortas**: de 6,8 años a 2,0. Bajar por la matriz es optimizar con cada vez menos historia.

2.4 Las cuatro familias de condiciones, con el juguete

SQX ofrece cuatro maneras de preguntar a una celda. Las diez condiciones de la tarea son de estas cuatro (`core/wfmobjectives.py:11-12`). Añado al juguete lo que ganó cada ventana de optimización:

```
tramo                      1       2       3       4       5     suma
optimización (5 años)   1.500   1.200   1.400   1.000     900    6.000   en 25 años-ventana
pasada (1 año)           +300    −100    +200     +50     −50     +400   en 5 años
```

Familia «OOS». La métrica del cosido, tal cual. *Beneficio OOS = +400. ¿Mayor que 0? Sí.*

Familia «estabilidad». Lo de fuera dividido entre lo de dentro, en porcentaje: cuánto de lo que prometía la optimización se cobró después. Para las métricas que crecen con el tiempo (beneficio, operaciones) se divide antes entre los días, porque 5 años de optimización no se pueden comparar con 1 de pasada:

```
fuera, por año:   400 / 5  =  80
dentro, por año:  6.000 / 25 = 240
estabilidad del beneficio = 80 / 240 × 100 = 33 %
```

Se cobró un tercio del ritmo prometido. Para el PF y el drawdown en porcentaje, que no crecen con el tiempo, se suman los de los tramos sin dividir entre días (`wfmobjectives.py:17-20`, `82-92`).

Familia «score». La métrica del walk-forward entero dividida entre la del backtest original, en porcentaje. Pregunta si reoptimizar deja la estrategia parecida a como era. El detalle de qué es «el walk-forward entero» es el punto 3 de la portada (3.3).

Familia «especiales». Cuentas sobre las pasadas una a una. SQX tiene siete (`wfmobjectives.py:13`); la tarea usa estas cuatro:

```
% de pasadas rentables       = 3 de 5 (los años 6, 8 y 9)          = 60 %
mayor beneficio de una pasada = 300 / 400 del total                  = 75 %
mínimo de operaciones en una pasada, máximo drawdown % de una pasada: el peor de los cinco
```

El 75 % dice que tres cuartas partes de lo ganado vinieron de un solo año: el resultado depende de un tramo.

2.5 De diez condiciones a «la celda pasa»

SQX cuenta cuántas condiciones activas se cumplen, lo pasa a porcentaje, **redondea al entero** (el medio hacia arriba) y compara con el umbral, 80 (`measure/gaterule.py:18-30`):

```
cumple 8 de 10  →  80 %   → pasa
cumple 7 de 10  →  70 %   → no pasa
con 9 condiciones activas: 7 de 9 = 77,8 → 78 → no pasa;  8 de 9 = 88,9 → 89 → pasa
```

Con diez condiciones el redondeo no interviene: pasar es **cumplir 8 o más**. Las diez pesan igual: fallar «beneficio mayor que cero» cuesta lo mismo que fallar «más de 20 operaciones por pasada».

2.6 La regla del área

Que pase una celda suelta no dice nada: con 30 troceos alguno saldrá bien. SQX pide que pasen **celdas vecinas**: que exista un rectángulo de 4 filas × 4 columnas con al menos 12 de sus 16 celdas aprobadas. La idea es buena: si el resultado aguanta al mover un poco el troceo, no depende de un troceo afortunado.

Juguete con una matriz de 4 × 3, rectángulos de 2 × 2 y un mínimo de 3 (1 = la celda pasa):

```
        p=20  p=30  p=40
R=4       1     1     0
R=6       1     0     0
R=8       0     1     1
R=10      0     1     1

rectángulos de 2×2 posibles: (4−2+1) × (3−2+1) = 3 × 2 = 6

filas R=4,6   · columnas 20,30:  1+1+1+0 = 3   ✓
filas R=4,6   · columnas 30,40:  1+0+0+0 = 1
filas R=6,8   · columnas 20,30:  1+0+0+1 = 2
filas R=6,8   · columnas 30,40:  0+0+1+1 = 2
filas R=8,10  · columnas 20,30:  0+0+1+1 = 2
filas R=8,10  · columnas 30,40:  1+1+1+1 = 4   ✓  (el mejor)
```

Pasa, porque algún rectángulo llega a 3. En la tarea real la matriz es de 6 × 5 y el rectángulo de 4 × 4, así que hay (6−4+1) × (5−4+1) = **3 × 2 = 6 posiciones** (`wfm.py:106`). Dos consecuencias de esa geometría:

- las columnas del medio (25, 30 y 35 %) están en **todas** las posiciones, y las filas de 10 y 12 tramos también. Una estrategia que falle en el centro de la matriz no puede pasar, haga lo que haga en los bordes;
- 12 de 16 es 75 %. Si las celdas fueran independientes sería un listón alto; no lo son (2.8).

SQX nombra además una **celda recomendada**: el centro del mejor rectángulo, una fila y una columna hacia dentro desde su esquina (`gaterule.py:81`). Si dos rectángulos empatan gana el primero leyendo de arriba abajo y de izquierda a derecha.

2.7 La ρ de Spearman: «¿lo que optimiza bien va bien después?»

Es la pregunta del estudio de Python. Se contesta dentro de cada celda, con sus tramos.

Correlación por rangos. Se ordenan los tramos de peor a mejor según la métrica de la optimización, y por separado según la métrica de la pasada. Si los dos órdenes se parecen, ρ se acerca a +1; si son opuestos, a −1; si no tienen que ver, a 0. Se usan rangos y no los valores para que un tramo disparatado no mande.

Con el juguete (métrica: beneficio):

```
tramo                    1       2       3       4       5
optimización         1.500   1.200   1.400   1.000     900
rango (1 = el peor)      5       3       4       2       1
pasada                +300    −100    +200     +50     −50
rango                    5       1       4       3       2
diferencia d             0       2       0      −1      −1
d²                       0       4       0       1       1      suma = 6

ρ = 1 − 6 × suma / (n × (n² − 1)) = 1 − 6 × 6 / (5 × 24) = 1 − 0,30 = +0,70
```

Aquí los tramos que optimizaron mejor fueron también los mejores fuera: ρ = +0,70.

Cuánto vale una ρ con cinco puntos. Poco. Con 5 tramos, por puro azar ρ sale mayor que +0,70 o menor que −0,70 una de cada cuatro o cinco veces; con 16 tramos la banda del azar es de ±0,50 aproximadamente. Por eso el estudio no lee ninguna celda sola: **junta las 30**.

Cómo junta. La ρ no se puede promediar directamente (cerca de ±1 se comprime). Se pasa cada ρ por una transformación que la estira (z de Fisher, `arctanh`), se promedian las 30 z, y se deshace la transformación (`measure/correlation.py:36-65`):

```
tres celdas de juguete:   ρ = +0,70    +0,10    −0,30
z = arctanh(ρ):              0,867     0,100    −0,310      media = 0,219
ρ conjunta = tanh(0,219) = +0,216
```

El intervalo. Para saber cuánto fiarse de la ρ conjunta el estudio hace un *bootstrap sobre las celdas*: saca 30 celdas al azar con repetición de las 30 que hay, calcula la ρ conjunta, y lo repite 9.999 veces. El 2,5 % y el 97,5 % de esas 9.999 son el intervalo.

El veredicto (`verdict/call.py:33-38`):

el intervalo	veredicto	lectura
entero por encima de 0	**predice**	los tramos que optimizan mejor van mejor después
contiene el 0	**ciega**	optimizar bien no dice nada del tramo siguiente
entero por debajo de 0	**perversa**	los tramos que optimizan mejor van peor después

Dos trampas de esta medida, que hay que conocer antes de usarla.

La primera, y la principal: el intervalo es demasiado estrecho. El bootstrap trata las 30 celdas como 30 medidas independientes. No lo son: las 30 celdas trocean **la misma historia**, y un mal 2022 lo es en todas. El propio código lo reconoce: el docstring de `pooled` dice que el intervalo es «un suelo de la incertidumbre» (`measure/correlation.py:56-65`). Cuánto se queda corto está medido en simulación (5.4): lo que de verdad varía la ρ conjunta de una historia a otra es **entre 4,2 y 5,2 veces** el error que da el bootstrap, como si las 30 celdas fueran entre 1,1 y 1,7. Un intervalo calculado sobre 30 no tiene sentido; la vara válida es la banda de la simulación sin ventaja, o un bootstrap por bloques de calendario que no existe todavía (5.14).

La segunda: la ρ tira hacia abajo sin que haya nada perverso. Mira otra vez el dibujo de 2.1: la ventana de optimización del tramo 2 **contiene** el año que se operó en el tramo 1. Si el año 6 fue muy bueno, la pasada del tramo 1 sale alta **y** la optimización del tramo 2 sale alta (lo lleva dentro). Es decir: un tramo con buena pasada tiende a ir seguido de un tramo con buena optimización, no al revés. Con pocos tramos, eso desplaza la correlación entre «optimización de este tramo» y «pasada de este tramo» hacia el lado negativo aunque la estrategia no tenga ventaja ni defecto alguno. Es un sesgo conocido de las regresiones con ventanas solapadas y pocos puntos, y aquí está demostrado en simulación: si la optimización se lee sobre una serie independiente (sin solape con las pasadas), el centro de la ρ pasa de −0,20 a 0,00. Cuánto pesa está en 5.4. Es la segunda trampa en importancia: quitarla no arregla la primera.

Qué pregunta contesta de verdad. No contesta «¿las configuraciones que mejor optimizan son las que mejor van?». Para eso habría que guardar todos los candidatos de cada optimización, y SQX sólo guarda al ganador. Contesta: «a lo largo del tiempo, ¿los periodos en que el ganador sacó una nota alta dentro fueron seguidos de periodos buenos fuera?». Es una pregunta sobre **épocas**, no sobre parámetros.

2.8 Cuántas celdas hay de verdad

Dos celdas vecinas (10 × 30 y 12 × 30, pongamos) operan casi los mismos años, con la misma estrategia y parámetros parecidos. Sus curvas diarias se parecen. Si dos curvas son idénticas, tener las dos no da más información que tener una.

El **número efectivo de celdas** resume eso en una cifra. Se calcula con la correlación entre los resultados diarios de las 30 curvas:

```
tres celdas idénticas (correlación 1 entre todas):        efectivas = 1
tres celdas sin relación (correlación 0 entre todas):     efectivas = 3
tres celdas con correlación 0,6 entre cada par:           efectivas ≈ 1,7
```

La cuenta exacta usa los «autovalores» de la matriz de correlaciones, que son una forma de repartir la variación total entre direcciones independientes: si casi toda la variación está en una sola dirección, hay una sola celda. Con autovalores λ:

```
efectivas = (suma de λ)² / (suma de λ²)

juguete con tres celdas y λ = 2,2 · 0,4 · 0,4:   (3,0)² / (4,84 + 0,16 + 0,16) = 9 / 5,16 = 1,74
```

Este número **no lo calcula el código**: lo calculé yo para el dossier del 2026-10-01 con el oro y lo he vuelto a calcular para esta corrida (3.8). Sale entre 2,2 y 4,6.

Es una definición entre varias, no una medida exacta. Si la pregunta es «¿cuánto baja la incertidumbre de una media al promediar las 30?», la misma correlación da entre 1,6 y 2,4; y para la ρ conjunta, la simulación de 5.4 da entre 1,1 y 1,7. Quédate con el orden de magnitud —un puñado, no treinta— y no con el decimal.

Qué le hace eso a la regla del área. Simulación ilustrativa mía: cada celda pasa con la probabilidad que se indica, y las celdas se parecen entre sí lo que dice la columna. La tabla da la probabilidad de que una estrategia pase la regla «4×4 con ≥ 12» **sin más mérito que esa probabilidad por celda**:

probabilidad de pasar por celda	celdas independientes	parecido 0,5	parecido 0,8
34 % (la tasa real de la corrida: 103 de 300)	0,4 %	19 %	27 %
50 %	13 %	38 %	44 %
60 %	40 %	51 %	55 %

Lo que la tabla enseña: la regla del área es muy sensible a cuánto se parecen las celdas. Con celdas independientes y un 34 % por celda, pasarla sería casi imposible por azar (0,4 %); en cuanto las celdas aprueban o suspenden juntas, la misma regla la pasa una parte apreciable.

Lo que la tabla **no** enseña, y no hay que leer en ella: cuántas estrategias sin ventaja pasarían el área de verdad. Por dos razones. El 34 % es la tasa de **esta** población, ya filtrada por el paso 8; no es la de una estrategia sin ventaja, que nadie ha medido. Y el «parecido» del modelo es una correlación oculta entre *aprobar* una celda y aprobar otra, no la correlación de las curvas diarias (0,40 a 0,65): no se puede leer una columna con el dato de la otra. *Ilustrativo: un único factor común con distribución normal; la aritmética está comprobada por el revisor, la aplicación a la corrida real no tiene apoyo.*

2.9 Qué es fuera de muestra aquí, y qué parte de `oos2` se lee

El OOS de la WFM lo es **respecto a la reoptimización de parámetros**: los parámetros de cada pasada se eligieron sin ver esa pasada. Pero la estrategia —sus reglas, sus indicadores, que haya llegado hasta aquí— se eligió antes:

- el generador la construyó mirando `build` (2011-2019);

- el paso 8 la dejó pasar mirando `oos1` (2020-2023);
- nadie ha elegido nada mirando `oos2` (2024-2026).

La celda 6 × 20 de C, tramo a tramo, con el tramo del activo en que cae cada pasada:

tramo	optimiza	opera	la pasada cae en
1	2011-09-19 → 2018-07-13	2018-07-14 → 2019-11-25	`build`
2	… → 2019-11-25	2019-11-26 → 2021-04-08	`build` / `oos1`
3	… → 2021-04-08	2021-04-09 → 2022-08-21	`oos1`
4	… → 2022-08-21	2022-08-22 → 2024-01-03	`oos1`
5	… → 2024-01-03	2024-01-04 → 2025-05-17	**`oos2`**
6	… → 2025-05-17	2025-05-18 → 2026-09-30	**`oos2`**

El tramo 4 está rotulado `oos1` pero acaba el 2024-01-03: sus tres últimos días son ya de `oos2`. En esta celda, 2 de 6 pasadas son `oos2`. En la de 16 × 40 el OOS empieza en 2013: 6,3 años de «fuera de muestra» están dentro de `build`.

Qué parte de `oos2` lee el paso: toda. Las 30 celdas terminan el 2026-09-30, así que el OOS cosido de cada una contiene los 2,75 años enteros. En días, `oos2` es el 33,5 % del cosido en la celda 6 × 20 y el 21 % en la 16 × 40; en operaciones, entre el 18 y el 44 % según la estrategia (C: 32 %).

Qué gasta. Gastar un tramo es usarlo para elegir. Hoy:

quién mira **`oos2`**	para qué	¿elige algo?
la regla del área de SQX	marcar la estrategia	no: la marca no borra
la ρ del estudio (2 de 6 a unos 4 de 16 tramos por celda)	veredicto predice / ciega / perversa	**sobre el papel**: es una pieza del paso 20, que marca DESCARTAR; el autopiloto no lo aplica, sólo `/curate` a mano
la pestaña «Contra el activo y el azar» (sólo `oos2`, la original)	enseñar	no
el ledger	tres filas del paso 19: `build`, `oos1`, `oos2`	anota la mirada

Hay un segundo uso, más callado: **las optimizaciones de los últimos tramos se hacen con datos de `oos2`**. La ventana del tramo 6 de la tabla llega hasta mayo de 2025, y la del tramo «futuro» hasta el último día. Mientras se siga adelante con la estrategia original, eso no gasta nada. Si algún día se adoptaran los parámetros «recomendados» de la WFM, `oos2` habría pasado a ser dentro de muestra para ellos.

Tres consecuencias de que entre el 67 y el 79 % del «OOS» sea `build` y `oos1`. Una: las condiciones C1, C2 y C7 (beneficio, PF y pasadas rentables del cosido) están infladas por selección, porque el paso 8 eligió a estas estrategias por ir bien en `oos1` (3.8 lo enseña tramo a tramo). Dos: la única evidencia limpia del paso es la pestaña del azar, que mide la original en `oos2` y **no necesita la WFM** para existir. Tres: tal como está, el paso gasta miradas a `oos2` en un veredicto —el de la ρ— que sin ventaja alguna sale «perversa» entre el 64 y el 74 % de las veces (5.4).

El día que se escriba una regla sobre este paso, `oos2` queda gastado para lo que esa regla elija, y la puerta del ledger (`pipeline/autopilot/run.py:163-164`) cuenta la mirada. Ojo con el alcance de esa puerta: la reserva sólo obliga a las corridas autónomas (`ALGO_AUTONOMOUS=1`).

2.10 Contra el activo y contra el azar

Son las dos comparaciones que pediste el 2026-10-01, y se hacen con la estrategia **original** (parámetros fijos, no una celda), sólo en `oos2`.

Contra el activo. Comprar el NIKKEI225 en enero de 2024 y no tocarlo también ganó dinero. La pregunta es si la estrategia aporta algo sobre eso. Tres números:

- **Diferencia de Sharpe:** Sharpe de la estrategia menos Sharpe del activo, los dos sobre resultados diarios y anualizados (media / desviación × √252). Juguete: estrategia 0,40, activo 1,14 → diferencia −0,74.
- **Beta:** cuánto se mueve la estrategia cuando el activo se mueve 1. Beta 0,03 quiere decir que casi nada: no es el activo disfrazado.
- **Sharpe del residuo:** se le quita a la estrategia la parte que explica el activo (beta × activo) y se calcula el Sharpe de lo que queda. Es «lo suyo propio».

Cada uno lleva su intervalo, hecho con un bootstrap **por bloques**: en vez de barajar días sueltos (que rompería las rachas) se barajan bloques de 20 días seguidos de media, 2.000 veces, sacando a la vez el día de la estrategia y el del activo para no romper su relación (`measure/benchmark.py:44-77`). Lectura: intervalo de la diferencia entero sobre 0, la estrategia bate al activo; entero bajo 0, pierde contra él; si contiene el 0, **no se distingue** (`benchmark.py:80-84`).

Contra el azar. Se construyen 10.000 traders que hacen **las mismas operaciones** que la estrategia —mismo número, misma duración, mismo tamaño, mismos costes— pero entrando en momentos sorteados (`measure/monkey.py`, peldaño `timing`). Si la estrategia sabe *cuándo* entrar, tiene que quedar por encima de casi todos:

```
juguete: de 10.000 traders al azar, 1.930 sacan un Sharpe por operación igual o mejor que el
de la estrategia
p = 1.930 / 10.000 = 0,193    →  uno de cada cinco traders al azar la iguala: no se distingue
del azar
```

El corte que usa la pestaña para pintar es 0,05 (`assetview.py:15`). Como el activo subió, entrar largo al azar también ganó: el trader al azar medio no queda a cero, y por eso esta comparación es más exigente que «¿gana dinero?».

---

3. Lo que calculan SQX y el estudio, pieza a pieza

Todo lo que sigue está recalculado desde el export. «✓» quiere decir que mi cuenta coincide con el valor guardado (por SQX en `conditions` / `objectives`, o por el estudio en `verdict.csv` y en los JSON de `estrategias/`).

3.1 La geometría: `model/windows.py`

salida	valor en la corrida	cómo se lee
tramos reales por estrategia	330 (329 en dos: 5.13)	5 × (6+8+10+12+14+16)
tramos futuros descartados	300 de 3.600	uno por celda
pasadas que se pisan dentro de una celda	0	el cosido no cuenta ningún día dos veces
solape entre ventanas de optimización seguidas	80 % con p = 20 … 60 % con p = 40; media 69,9 %	es 1 − p: cuanto más corta la pasada, más se repite la ventana
historia cubierta por algún OOS	13,01 años (2013-09 → 2026-09)	la unión de las 30 celdas

El README del estudio dice «las ventanas de optimización se solapan un 79 %»: es el caso de p = 20, no la media (5.12).

3.2 Las diez condiciones, sobre una celda real

Celda **6 × 20 de C** (`14.3.479`). Sus seis tramos:

tramo	beneficio IS	beneficio OOS	PF IS	PF OOS	DD % IS	DD % OOS	ops IS	ops OOS
1	25.956,18	−3.003,98	2,45	0,63	2,54	5,01	150	34
2	18.385,92	+5.938,87	2,10	2,50	2,06	1,53	152	35
3	23.268,66	−2.867,14	2,35	0,46	1,80	3,42	160	37
4	15.740,42	−1.038,38	1,80	0,82	2,08	1,87	163	41
5	18.187,89	+2.485,02	1,64	1,28	2,40	2,32	203	63
6	24.557,36	+5.126,76	1,49	1,48	4,02	3,63	328	75
suma	126.096,43	**+6.641,15**	11,83	7,17	14,90	17,78		

Cada ventana de optimización dura 2.489 días (seis: 14.934); cada pasada 499 o 500 (seis: 2.995). Las diez condiciones, en el orden de `assets/_build.yaml:550-562` :

nº	familia	condición	cuenta	valor	¿cumple?
C1	OOS	beneficio neto > 0	suma de las seis pasadas	6.641,15	sí ✓
C2	OOS	profit factor > 1,05	PF del cosido (de sus 285 operaciones)	1,15	sí ✓
C3	estabilidad	PF > 60	7,17 / 11,83 × 100	60,61	sí ✓ (por 0,61)
C4	estabilidad	beneficio neto > 20	(6.641,15 / 2.995) / (126.096,43 / 14.934) × 100 = 2,217 / 8,444 × 100	26,26	sí ✓
C5	estabilidad	drawdown % < 150	17,78 / 14,90 × 100	119,33	sí ✓
C6	score	PF > 85	1,54 / 1,28 × 100 (ver 3.3)	120,31	sí ✓
C7	especial	% de pasadas rentables > 50	3 de 6	50,0	**no** ✓
C8	especial	mayor beneficio de una pasada < 50 %	5.938,87 / 6.641,15	89,43	**no** ✓
C9	especial	mínimo de operaciones en una pasada > 20	la menor de 34, 35, 37, 41, 63, 75	34	sí ✓
C10	especial	máximo drawdown % de una pasada ≤ 25	el mayor de la columna DD % OOS	5,01	sí ✓

Cumple 8 de 10 → 80 % → **la celda pasa, justa**. Cuatro notas sobre las fórmulas, cada una con su línea:

- **C7 es «mayor que», no «mayor o igual»** (`_build.yaml` l.550-562). Tres de seis es exactamente 50 y no cumple. En las filas de 6, 8, 10… tramos, que son todas pares, «la mitad justa» es un resultado posible y siempre cuenta como fallo.
- **C8 devuelve 100 cuando el total del OOS es negativo** (`core/wfmobjectives.py:107`). Es un valor centinela, no una medida: 105 de las 300 celdas tienen 100 por esa razón, y por eso la mediana de C8 es exactamente 100,0.
- **La estabilidad se calcula sin el último tramo** (`wfmobjectives.py:82-92`, `steps[:-1]`): el que se quita es el futuro. Está bien; lo digo porque la etiqueta de la ventana («sin el último tramo», `labels.py:15-17`) se puede leer como que se pierde un tramo real.
- **La estabilidad del PF y del drawdown % suma los valores de los tramos sin ponderar** (`wfmobjectives.py:17-20`). Una pasada con 34 operaciones pesa lo mismo que una con 75.

3.3 El «score», con su numerador

`core/wfmobjectives.py:133-134` y `152`: score = métrica de la muestra «todo» (127) del walk-forward ÷ métrica de la muestra «todo» del backtest original × 100. La muestra «todo» del walk-forward es **la primera ventana de optimización más el OOS cosido** (2.2). En la celda 6 × 20 de C:

```
primera ventana optimizada:  150 operaciones,  +25.956
OOS cosido:                  285 operaciones,   +6.641
«todo»:                      435 operaciones,  +32.597     PF 1,54
backtest original, 15 años:  402 operaciones,  +18.309     PF 1,28

score PF = 1,54 / 1,28 × 100 = 120,31   ✓
con sólo el OOS:  1,15 / 1,28 × 100 = 89,8
```

Que el numerador es la muestra 127 no descansa en esta celda sola: lo apoya `core/wfmatrix.py:66-70` y lo confirman las 300 celdas (el revisor reprodujo el score de todas; el cociente entre el PF «todo» y el score es constante por estrategia e igual a su PF original: 1,5299-1,5301 en `10.11.484`, 1,6299-1,6301 en G).

El 80 % del beneficio del numerador (25.956 de 32.597) es el resultado de una optimización leído sobre los mismos años que se optimizaron. La condición acaba diciendo «el walk-forward sale un 20 % mejor que la original» de una celda cuyo OOS tiene un PF de 1,15 frente al 1,28 de la original.

Cuánto pesa en toda la corrida (cálculo mío): la primera ventana aporta de media el 21 % de las operaciones de «todo» (entre el 4 y el 52 % según la celda). Con sólo el OOS en el numerador, C6 la cumpliría el **34 %** de las celdas en vez del **70,7 %**, y la regla del área la pasarían 2 de 10 (C con 12 y G con 16) en vez de 3: E bajaría de 12 a 9.

estrategia	PF original	score mediano tal como es	con sólo el OOS
`10.11.484`	1,53	78,4	72,9
`13.3.390`	1,31	89,3	74,8
C	1,28	91,8	85,5
F	1,06	99,1	92,9
`16.5.352`	1,07	114,0	88,3
`18.5.355`	1,35	88,1	75,2
`19.2.363`	1,03	98,5	89,8
E	1,49	90,9	76,5
`3.9.284`	1,35	84,8	78,1
G	1,63	84,4	78,2

Segunda cosa que enseña la tabla: **el score premia a la original floja**. Las tres estrategias con PF original más bajo (1,03, 1,06, 1,07) tienen los scores más altos, y `19.2.363` —la peor de las diez, 0 celdas aprobadas— cumple C6 en 27 de 30. Dividir entre un PF de 1,03 hace fácil llegar a 85.

3.4 Las diez condiciones en las 300 celdas

Pestaña «Qué condición manda» (`contract.py`). La última columna es el porcentaje que el YAML anota al lado de cada condición: cuántas celdas la cumplían cuando se calibró, con 150 celdas de XAUUSD del master.

nº	condición	mediana de las 300	cumplen aquí	cumplían en el oro
C1	OOS beneficio > 0	8.768	64,7 %	66 %
C2	OOS PF > 1,05	1,045	47,7 %	45 %
C3	estabilidad PF > 60	66,77	80,0 %	80 %
C4	estabilidad beneficio > 20	10,87	40,3 %	30 %
C5	estabilidad DD % < 150	158,05	**42,0 %**	**79 %**
C6	score PF > 85	89,74	70,7 %	83 %
C7	% pasadas rentables > 50	53,1	50,0 %	51 %
C8	mayor beneficio de una pasada < 50	100,0 (centinela)	**26,0 %**	**11 %**
C9	mínimo de operaciones > 20	29	68,3 %	55 %
C10	máximo DD % de una pasada ≤ 25	13,27	97,0 %	100 %

Lecturas:

- **C10 no discrimina** (97 %): con el tamaño de posición de la tarea ninguna pasada se acerca al 25 %. Es una condición regalada, y al estar en el denominador convierte «8 de 10» en «7 de las 9 que cuentan».
- **C3 casi tampoco** (80 %).
- **C8 es la que más tumba** (26 %), y casi la mitad de sus fallos (105 de 222) son el centinela de 3.2, que repite a C1: una celda con OOS negativo falla C1 **y** C8 por la misma razón. Las condiciones no son diez preguntas distintas: C1, C2, C4, C7 y C8 miran todas el beneficio del cosido desde ángulos cercanos.
- **C5 se cumple aquí la mitad de veces que en el oro.** No es que estas estrategias sean peores: es que el umbral se puso con otro activo y otra ventana.

Cuántas condiciones cumple cada celda (las 300): 0 → 1 · 1 → 10 · 2 → 20 · 3 → 46 · 4 → 36 · 5 → 29 · 6 → 22 · 7 → 33 · **8** → **38 · 9** → **34 · 10** → **31**. Pasan (8 o más) **103 de 300**, el 34 %. Hay 33 celdas que se quedan en 7: a una condición.

Celdas (de 30) que cumplen cada condición, por estrategia:

estrategia	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	pasan
`10.11.484`	29	21	29	20	8	4	22	13	29	30	13
`13.3.390`	11	2	30	2	17	26	5	1	22	28	2
C	25	21	25	16	16	23	20	9	13	30	13
F	14	8	30	8	14	25	11	2	17	26	8
`16.5.352`	10	6	13	6	2	30	6	5	25	29	5
`18.5.355`	18	11	22	6	19	21	15	3	21	28	9
`19.2.363`	3	1	12	1	8	27	1	1	1	30	0
E	30	28	22	22	16	27	25	12	21	30	17
`3.9.284`	24	15	27	11	6	15	17	2	28	30	9
G	30	30	30	29	20	14	28	30	28	30	27

`10.11.484` gana dinero fuera de muestra en 29 de 30 celdas y no pasa el área: se lo impiden C5 (8 de 30) y C6 (4 de 30). C6 falla porque su PF original es alto (1,53) y la reoptimización no lo iguala: la misma condición que regala el aprobado a la original floja se lo quita a la original buena.

3.5 De las celdas a la marca de SQX

`measure/gaterule.py:52-81`. Condiciones cumplidas por celda (de 10); filas = tramos, columnas = % OOS. En negrita las que pasan (8 o más).

C — 13 celdas pasan:

tramos	20	25	30	35	40
6	**8**	**8**	**8**	**10**	7
8	7	**10**	**10**	**9**	1
10	**10**	**9**	**9**	3	7
12	**8**	7	**8**	3	1
14	6	4	6	**8**	3
16	6	7	5	6	4

Las seis posiciones del rectángulo de 4 × 4:

```
filas 6-12,  columnas 20-35:   4 + 3 + 3 + 2 = 12   ✓  (el mejor)
filas 6-12,  columnas 25-40:   3 + 3 + 2 + 1 =  9
filas 8-14,  columnas 20-35:   3 + 3 + 2 + 1 =  9
filas 8-14,  columnas 25-40:   3 + 2 + 1 + 1 =  7
filas 10-16, columnas 20-35:   3 + 2 + 1 + 0 =  6
filas 10-16, columnas 25-40:   2 + 1 + 1 + 0 =  4
```

Llega a 12 en una sola posición y con el mínimo exacto → **pasa**. Celda recomendada: 8 tramos × 25 % (10 de 10). Fíjate en lo cerca que está: cinco de las 12 celdas aprobadas del rectángulo tienen exactamente 8, y la 6 × 20 —la de 3.2— cumple C3 por 0,61 puntos. Si esa celda bajara a 7, el rectángulo tendría 11 y C

suspendería.

F — 8 celdas pasan; mejor rectángulo 8 → no pasa:

tramos	20	25	30	35	40
6	**9**	**9**	**9**	**9**	5
8	**10**	**10**	6	3	2
10	**9**	4	4	**8**	4
12	5	1	3	4	4
14	6	6	3	3	3
16	5	2	3	2	4

E — 17 celdas pasan; mejor rectángulo 12 → pasa, justo:

tramos	20	25	30	35	40
6	**10**	**8**	**10**	**10**	**8**
8	**10**	**10**	**9**	**8**	7
10	**9**	7	7	**8**	**9**
12	**10**	**9**	7	5	4
14	7	7	**8**	7	6
16	**8**	**9**	6	7	3

G — 27 celdas pasan; mejor rectángulo 16 de 16 → pasa:

tramos	20	25	30	35	40
6	**10**	**10**	**10**	**10**	**9**
8	**10**	**9**	**9**	**10**	**8**
10	**9**	**10**	**9**	**10**	**8**
12	**10**	**10**	**9**	**9**	**8**
14	**9**	**9**	**10**	7	**8**
16	**9**	**8**	**8**	7	7

En las cuatro se ve el mismo dibujo: lo bueno está arriba (pocos tramos, ventanas de optimización largas) y se estropea hacia abajo (ventanas de 2 a 3 años). No es una propiedad de estas estrategias; es lo que le pasa a cualquier optimización cuando se le da menos historia.

Las diez: las celdas que pasan y el mejor rectángulo están en la tabla de 1.7. **Mi reconstrucción da el mismo resultado que la marca de SQX en las diez: pasan C, E y G; las otras siete llevan «FAILED»** y siguen en el databank.

3.6 La ρ: `measure/correlation.py` y `verdict/call.py`

Métrica: `ReturnDDRatio` (beneficio dividido entre drawdown), la de `config.yaml` `read.metric`. Para la celda 6 × 20 de C:

```
tramo                       1      2      3      4      5      6
RetDD de la optimización  9,90   8,94  11,98   6,92   6,56   5,92
rango (1 = el peor)          5      4      6      3      2      1
RetDD de la pasada       −0,60   3,81  −0,83  −0,55   1,05   1,36
rango                        2      6      1      3      4      5
d²                           9      4     25      0      4     16      suma = 58

ρ = 1 − 6 × 58 / (6 × 35) = 1 − 1,657 = −0,657     ✓ (el estudio: −0,66)
```

Las 30 ρ de C:

tramos	20	25	30	35	40
6	−0,66	+0,14	−0,49	+0,20	−0,03
8	−0,24	−0,10	+0,17	−0,48	+0,02
10	−0,21	+0,08	+0,33	+0,14	+0,52
12	−0,14	−0,12	−0,14	−0,05	+0,01
14	−0,04	−0,08	+0,20	−0,29	−0,16
16	−0,14	+0,07	+0,14	−0,52	+0,14

Media de las 30 z = −0,0645 → ρ conjunta = tanh(−0,0645) = **−0,064** ✓. 17 de 30 celdas negativas (57 %). Intervalo del bootstrap sobre celdas: [−0,163, +0,034] ✓ → contiene el 0 → **ciega**. (La desviación de las 30 z es 0,284; 0,284 / √30 = 0,052; ±1,96 × 0,052 = ±0,10, que es el ancho del intervalo: el bootstrap está haciendo, en la práctica, la cuenta de «30 medidas independientes».)

Las diez (✓ contra `verdict.csv`):

estrategia	ρ	intervalo	celdas negativas	veredicto	pieza en el paso 20
`10.11.484`	−0,021	[−0,116, +0,074]	47 %	ciega	vigilar
`13.3.390`	−0,133	[−0,258, −0,018]	57 %	perversa	fallo
C	−0,064	[−0,163, +0,034]	57 %	ciega	vigilar
F	**+0,345**	[+0,273, +0,411]	3 %	**predice**	bien
`16.5.352`	+0,018	[−0,066, +0,099]	37 %	ciega	vigilar
`18.5.355`	−0,194	[−0,308, −0,067]	80 %	perversa	fallo
`19.2.363`	−0,158	[−0,269, −0,047]	67 %	perversa	fallo
E	−0,021	[−0,132, +0,088]	43 %	ciega	vigilar
`3.9.284`	−0,419	[−0,514, −0,319]	97 %	perversa	fallo
G	−0,254	[−0,382, −0,119]	73 %	perversa	fallo

Las 300 celdas juntas: ρ media −0,079, el 56 % negativas. Por filas (6 a 16 tramos): −0,13 · −0,11 · −0,06 · −0,11 · −0,02 · −0,04. La población entera está corrida hacia el lado negativo, más en las filas de pocos tramos. La simulación sin ventaja (5.4) da por filas entre −0,24 y −0,13; lo real es entre −0,13 y −0,02: mismo signo y misma pendiente, mitad de tamaño. La diferencia no está explicada.

La misma pregunta con otras métricas (pestaña «La misma correlación en otras métricas»; `run.py:120-146`):

estrategia	beneficio	PF	RetDD (la oficial)	Sharpe
`10.11.484`	perversa	ciega	ciega	ciega
`13.3.390`	perversa	**predice**	perversa	ciega
C	perversa	**predice**	ciega	perversa
F	predice	predice	predice	predice
`16.5.352`	predice	ciega	ciega	ciega
`18.5.355`	ciega	perversa	perversa	perversa
`19.2.363`	perversa	perversa	perversa	perversa
E	perversa	perversa	ciega	ciega
`3.9.284`	perversa	perversa	perversa	perversa
G	perversa	ciega	perversa	perversa

Sólo tres estrategias (F, `19.2.363`, `3.9.284`) dan el mismo veredicto con las cuatro métricas. C es «predice» con el PF (+0,14) y «perversa» con el beneficio (−0,19). Un veredicto que cambia de signo al cambiar la vara de medir no está midiendo una propiedad firme de la estrategia.

3.7 La deriva de los parámetros: `measure/drift.py`

Pregunta: de un tramo al siguiente, ¿cuántos parámetros cambia la reoptimización? Si casi todos cambian cada vez, no hay un «valor bueno» estable: la optimización persigue el ruido de cada ventana.

`share_changed` = parámetros que cambian ÷ columnas (`drift.py:30-36`), y de ahí la mediana sobre todos los pares de tramos seguidos. Si esa mediana llega a 0,5 el estudio pone el aviso «deriva alta» (`verdict/call.py`, `config.yaml` `drift_high: 0.5`). Celda 6 × 20 de C: entre sus siete elecciones hay seis saltos, y cambian 15, 0, 13, 14, 15 y 14 columnas.

```
lo que hace el código:   15 / 19 = 0,789     ✓ (el valor guardado de C)
C tiene 17 parámetros:   15 / 17 = 0,882
```

Las 19 columnas son los 17 parámetros de C **más** `oos_pct` **y** `runs`, que el lector deja como columnas (`inputs/export.py:60-73`) y que dentro de una celda no cambian nunca. El denominador está inflado en dos. Las diez, con el código y corregido:

estrategia	parámetros	cambian (código)	cambian (sin las dos columnas)
`10.11.484`	9	0,727	0,889
`13.3.390`	10	0,667	0,800
C	17	0,789	0,882
F	15	0,765	0,867
`16.5.352`	6	0,625	0,833
`18.5.355`	7	0,667	0,857
`19.2.363`	6	0,625	0,833
E	10	0,750	0,900
`3.9.284`	10	0,750	0,900
G	13	0,800	0,923

El aviso «deriva alta» sale en las diez de las dos maneras, así que el error no cambia ningún veredicto de esta corrida. Pero el dato es más duro de lo que se enseña: **en un salto típico la reoptimización mueve entre 8 y 9 de cada 10 parámetros**, incluida F, la única que «predice». Un aviso que se enciende en las diez tampoco distingue nada.

Otros dos detalles: los saltos incluyen la elección del tramo futuro (es la excepción de 1.4: aquí sí se usa), y `summary.parameters` vale 74 para las diez estrategias (`drift.py:59`): cuenta las columnas de la tabla entera —los 72 parámetros distintos de las diez juntas, más las dos de arriba—, no los de cada una.

3.8 Las 30 celdas como población: `measure/equity.py`

Pestaña «El conjunto de las celdas». Para C:

métrica del OOS cosido	mediana	cuartil bajo	cuartil alto	peor	mejor
beneficio neto	8.872	3.400	11.396	−13.319	19.212
profit factor	1,09	—	—	0,92	1,32
Sharpe	0,21	—	—	−0,21	0,56
drawdown máximo	9.696	—	—	25.503	4.672
operaciones	563	—	—	285	1.050

83 % de las celdas en beneficio (25 de 30). Las diez, con las dos columnas que **no** calcula el código (parecido entre celdas y celdas efectivas; cálculo mío sobre los resultados diarios de los 30 cosidos, 2.8):

estrategia	celdas en beneficio	Sharpe mediano	beneficio mediano	parecido mediano entre celdas	celdas efectivas
`10.11.484`	97 %	0,30	22.130	0,57	2,7
`13.3.390`	37 %	−0,06	−4.975	0,57	2,8
C	83 %	0,21	8.872	0,65	**2,2**
F	47 %	−0,04	−2.747	0,45	3,9
`16.5.352`	33 %	−0,15	−10.969	0,41	4,6
`18.5.355`	60 %	0,05	4.497	0,40	**4,6**
`19.2.363`	10 %	−0,18	−6.817	0,56	2,8
E	100 %	0,33	27.216	0,58	2,5
`3.9.284`	80 %	0,13	10.716	0,49	3,5
G	100 %	0,62	54.606	0,63	2,3

Las tres que pasan el área (C, E, G) son las tres con **menos** celdas efectivas (2,2, 2,5, 2,3). Tiene sentido y es incómodo: cuanto más se parecen las celdas, más fácil es que pasen —o fallen— todas a la vez. Pasar 12 de 16 con 2,3 celdas efectivas es, por decirlo con una imagen, acertar dos veces y no doce. Es una figura, no una medida: «celdas efectivas» depende de la definición (2.8), y con la de la varianza de una media las diez quedan entre 1,6 y 2,4.

Dónde gana cada celda. Partiendo el cosido por los tramos del activo (cálculo mío):

estrategia	celdas en positivo en `build`	en `oos1`	en `oos2`	mediana `build`	mediana `oos1`	mediana `oos2`
`10.11.484`	37 %	93 %	93 %			
`13.3.390`	47 %	30 %	77 %			
C	63 %	83 %	90 %	+2.166	+2.690	+5.264
F	**0 %**	90 %	50 %	−23.373	+19.842	−184
`16.5.352`	33 %	43 %	37 %			
`18.5.355`	57 %	83 %	13 %			
`19.2.363`	10 %	20 %	47 %			
E	33 %	93 %	100 %	−6.352	+11.991	+19.620
`3.9.284`	53 %	90 %	83 %			
G	97 %	100 %	97 %	+16.167	+22.343	+15.315

`oos1` es el tramo en que casi todas van bien (siete de diez con el 83 % o más de las celdas en positivo). No sorprende: el paso 8 las eligió por ir bien ahí. F es el caso extremo: ninguna de sus 30 celdas gana en `build`, casi todas ganan en `oos1`, y en `oos2` es una moneda. La regla del área no separa estos tres tramos: suma los tres.

3.9 La pestaña «Contra el activo y el azar»: `measure/benchmark.py`, `measure/monkey.py`

La estrategia original, parámetros fijos, del 2024-01-02 al 2026-09-30 (710 días de mercado), sobre un capital de 100.000. El NIKKEI225 comprado tuvo en esa ventana un Sharpe de **1,14**.

estrategia	Sharpe	diferencia con el activo	intervalo	beta	Sharpe del residuo	operaciones (por año)	p del azar (Sharpe)
`10.11.484`	1,17	+0,03	[−1,28, +1,27]	0,05	0,89	101 (37)	**0,031**
`13.3.390`	0,78	−0,35	[−1,61, +0,81]	0,03	0,55	45 (16)	0,153
C	0,40	−0,74	[−1,99, +0,36]	0,03	0,21	115 (42)	0,193
F	0,43	−0,71	[−1,89, +0,53]	0,05	0,16	111 (40)	0,268
`16.5.352`	0,25	−0,88	[−2,11, +0,29]	0,11	−0,16	125 (45)	0,464
`18.5.355`	−0,24	−1,38	[−2,82, +0,11]	0,05	−0,53	77 (28)	0,729
`19.2.363`	−0,03	−1,17	[−2,42, +0,09]	0,04	−0,34	138 (50)	0,495
E	0,23	−0,91	[−2,06, +0,29]	0,05	−0,05	79 (29)	0,436
`3.9.284`	0,32	−0,81	[−2,00, +0,38]	0,06	0,03	142 (52)	0,347
G	0,93	−0,21	[−1,26, +0,92]	0,13	0,54	118 (43)	0,092

He recalculado el Sharpe de C (0,395) y de G (0,932) desde las operaciones: ✓. Cómo sale el de C:

```
resultado diario medio / 100.000 = 0,0000611      desviación diaria = 0,002454
Sharpe = 0,0000611 / 0,002454 × √252 = 0,0249 × 15,87 = 0,395
el 87,6 % de los días no tiene posición abierta
```

Lecturas:

- **Contra el activo, las diez son «no se distingue».** Ninguna lo bate y de ninguna se puede decir que pierda. Los intervalos miden ±1,2 de Sharpe: 2,75 años no dan para más. Nueve de las diez tienen un Sharpe menor que el del activo, pero el dato no llega a ser prueba.
- **Las betas son casi cero** (0,03 a 0,13): no son el índice disfrazado. Ganan o pierden por lo suyo.
- **Antes de leer las p: ninguna es utilizable hasta cerrar 5.10** (el motor del azar da a la estrategia un beneficio distinto del de SQX, sin explicar). Lo que sigue vale como orientación.
- **Contra el azar pasa una de diez** (`10.11.484`, p = 0,031), sin corregir por haber mirado diez. Con diez estrategias sin ventaja, que al menos una baje de 0,05 ocurre el 40 % de las veces. Tres estrategias (`13.3.390`, `18.5.355`, E) operan menos de las 35 veces por año de tu suelo.

- **Las tres que pasan el área de SQX** tienen p de 0,19 (C), 0,44 (E) y 0,09 (G). La que bate al azar no pasa el área.

Año a año (la estrategia frente al activo escalado a la misma volatilidad): C +7,50 % frente a +3,15 % en 2024, −1,10 frente a +4,33 en 2025, −2,06 frente a +5,01 en 2026. G +2,06 / +6,66, +15,20 / +9,15, +4,35 / +10,59.

El detalle del azar para C (10.000 traders):

estadístico	la estrategia	trader al azar medio	el 5 % mejor al azar	p
beneficio neto	7.021,62	1.532,85	14.203	0,239
Sharpe por operación	0,106	0,020	0,175	**0,193**
profit factor	1,406	1,149	—	0,231
beneficio / drawdown	1,884	0,777	—	0,186
drawdown	3.727,53	7.469,54	—	0,135

El trader al azar medio gana 1.533: entrar largo en cualquier momento en un índice que sube ya es rentable. C gana cuatro veces y media más, y aun así uno de cada cuatro o cinco traders al azar la iguala. Dos avisos sobre esta tabla: la semilla del sorteo es nueva en cada corrida (queda apuntada en la nota de la pestaña), así que la p no se repite al decimal; y el «7.021,62» no es el beneficio que da SQX para esas 115 operaciones, que es 4.337,65 (5.10).

3.10 Los hechos que saldrían, y los que no

Hoy el paso 19 no guarda hechos (4.1). He pasado el lector de hechos (`pipeline/autopilot/facts.py:73-109`) sobre la carpeta del informe, en memoria, para ver qué habría: **156 hechos numéricos por estrategia**.

grupo	hechos	ejemplo
`summary.*`	9	`rho` , `low` , `high` , `cells` , `steps` , `share_negative` , `share_changed` , `drift_high` , `sqx_failed`
`verdict.*`	4	`pass` (1 sólo si «predice»), `score` , celdas con ρ negativa, parámetros que cambian
«Qué condición manda»	10	la mediana de cada condición en las 30 celdas
«El conjunto de las celdas»	~26	mediana, cuartiles, peor y mejor de cada métrica; % de celdas en beneficio
«Otras métricas»	12	ρ, límites y celdas negativas para beneficio, Sharpe y PF
rejilla de objetivos	30	**sólo C1** (beneficio OOS), celda a celda: es el selector por defecto
«Contra el activo y el azar»	~55	diferencia de Sharpe, beta, residuo, año a año, tabla del azar, percentiles
`verdict_csv.*`	9	lo mismo que `summary`

Lo que no saldría como hecho, y haría falta para casi cualquier regla sobre la marca de SQX:

- cuántas celdas pasan (está como texto: «13 de 30»);

- cuántas tiene el mejor rectángulo, y si la regla del área pasa (sólo existe el negativo, `summary.sqx_failed`);
- cuántas condiciones cumple cada celda, y las condiciones C2 a C10 celda a celda;
- las celdas efectivas y el parecido entre celdas (no se calculan);
- qué parte del resultado de cada celda viene de `oos2` (no se calcula).

3.11 Lo que trae el export de SQX y el estudio no usa

La tabla `objectives` guarda todos los objetivos que SQX sabe calcular, no sólo los diez de la tarea. Para la celda 6 × 20 de C: estabilidad del Sharpe −2,73, estabilidad de operaciones 122, score del beneficio 178,04, score del Sharpe 181,08, score del drawdown % 72,77, estancamiento máximo 100 %, y un `param_stability` de 0,75 que SQX calcula a su manera. El score del beneficio (178) repite lo de 3.3 con más fuerza: «el walk-forward gana un 78 % más que la original» en una celda cuyo OOS gana 6.641 en ocho años.

---

4. El criterio cargado y por qué

4.1 Tal como está hoy en disco

`pipeline/autopilot/criteria.yaml` tiene entradas para los pasos 8, 10, 11.5, 12, 14 y 16. **No hay `"19"`.** La línea 59 explica por qué:

> PROVISIONAL (Council 2026-10-04): do not freeze before a null population has run the chain. Steps 17, 18, 19, 21, 23, 24 annotate only: no rule on purpose.

`ledger/thresholds.yaml` no tiene ninguna entrada de WFM. Los únicos umbrales del paso están en dos sitios: los de SQX en `assets/_build.yaml:447-565` (las diez condiciones, 80 %, 4 × 4, 12) y los del estudio en `studies/optimisation/wfm/config.yaml` (banda del cero 0,0, deriva alta 0,5, 9.999 remuestreos, 95 %; para la pestaña, 2.000 remuestreos, bloques de 20 días, 10.000 traders, mínimo 30 operaciones).

Y un escalón más abajo que el paso 16: **el piloto no juzga el paso 19**. En `pipeline/autopilot/tail.py:111-124` el paso 19 tiene cuatro acciones —la tarea de SQX, «lectura de la WFM», «guardar para fondeo» y el ledger— y ninguna es un juez; sólo `visit()` (l.89-99) programa uno, y lo hace para el paso 20 y para los pasos que se le pidan expresamente. Consecuencia comprobada: en `AlgoData/autopilot/Trade_NIKKEI225_M30_volatilidadL/` no existe ninguna carpeta `paso-19`. Lo que sí queda: el informe del estudio en `AlgoData/reports/…/WFM/`, tres filas en el ledger con el criterio «wfm/rho» y tantas estrategias de entrada como de salida (`ledger/steprows.py:61-83`), y un hecho en el paso 20, `blindJoint.verdict.19_wfm`, que es la ρ.

4.2 El único sitio donde el paso 19 pesa en algo: el paso 20, y sólo sobre el papel

`studies/optimisation/wfm/many.py:13` traduce el veredicto: predice → bien, ciega → vigilar, perversa → fallo. `studies/closing/blindJoint/readings.py:17` descarta si alguna pieza falla, que es la opción cargada (`blindJoint/config.yaml:26`, `pieces: sin_fallo`). En la corrida: la pieza WFM fue fallo en 5, vigilar en 4 y bien en 1. Las diez acabaron con DESCARTAR en el `verdict.csv` del paso 20, pero no por

esto: la pieza de superficies por mercado falló en las diez. Si esa pieza no hubiera fallado, **la WFM habría marcado a cinco, G entre ellas**, con el veredicto que 5.4 dice que sale solo entre el 64 y el 74 % de las veces.

Esa marca no corta nada por sí sola. `criteria.yaml` no tiene entrada para el paso 20; `pipeline/autopilot/judge.py:172-173` devuelve «ninguno: el paso no tiene criterio»; el 20 no tiene `cut`, y `blindJoint` no llama a `apply_verdict`. El autopiloto no aplica el DESCARTAR y todas siguen. Sólo corta si alguien lo aplica a mano con `/curate`. Es decir: hoy el paso 19 no elimina a ninguna estrategia por ninguna vía automática, pero deja escrita una recomendación de descarte que se puede ejecutar con una orden.

4.3 Lo que haría falta para escribir una regla

No propongo ninguna; apunto lo que falta para que tú puedas escribirla:

1. **Que el paso 19 guarde hechos**: que el piloto lo juzgue como hace con el 16 (hoy `tail.py` no lo hace).
2. **Que la marca de SQX sea un hecho en positivo y con números**: celdas que pasan, mejor rectángulo, pasa o no. Hoy sólo hay `sqx_failed`.
3. **Una población sin ventaja pasada por la WFM.** Sin ella no se sabe cuántas estrategias de azar pasan el área ni cuántas dan «perversa». Las dos simulaciones ilustrativas de este documento (2.8 y 5.4) dicen que muchas; no sustituyen a la medida.
4. **Decidir qué tramo cuenta.** Una regla sobre «la celda» mezcla `build`, `oos1` y `oos2`. Una regla sobre `oos2` solo tiene 2 de 6 tramos por celda.
5. **Saber cuántas celdas efectivas hay**, si la regla va a contar celdas.

4.4 Cuántas fallaría cada regla posible, sobre la población real

si la regla fuera…	fallan	quedan
la marca de SQX tal como está (área 4 × 4, ≥ 12, 80 %)	7 de 10	C, E, G
la misma, con el score calculado sólo con el OOS	8 de 10	C, G
«perversa» es fallo (lo que marca hoy el paso 20, sin aplicarlo)	5 de 10	`10.11.484`, C, F, `16.5.352`, E
las dos anteriores a la vez (marca de SQX y no perversa)	8 de 10	C, E
exigir «predice»	9 de 10	F
marca de SQX y «predice»	10 de 10	ninguna
p del azar ≤ 0,05 en `oos2`	9 de 10	`10.11.484`
batir al activo en `oos2` (intervalo sobre 0)	10 de 10	ninguna
no perder contra el activo en `oos2` (intervalo no entero bajo 0)	0 de 10	las diez

Ninguna estrategia pasa a la vez la marca de SQX y el azar. Las dos lecturas que da el propio paso —SQX y la ρ— coinciden sólo en C y E, y discrepan justo en las dos más llamativas: F y G.

4.5 Cómo sería el resultado ideal

Una estrategia con ventaja de verdad y parámetros que no importan mucho debería dar: casi todas las celdas en beneficio **en los tres tramos por separado**, no sólo sumados; un área que pase con holgura y no en una sola posición con el mínimo; condiciones cumplidas lejos del umbral; una ρ que no cambie de signo con la métrica; y, en `oos2`, la original por encima de casi todos los traders al azar. De las diez, **G** es la única que se acerca en las tres primeras (27 celdas, 16 de 16, 97-100 % de celdas en positivo en cada tramo) y falla en las otras dos (perversa con tres de cuatro métricas; p = 0,09).

4.6 Si mueves un umbral

Estrategias que pasan el área (de 10) según el porcentaje de condiciones que se pide a la celda y el mínimo de celdas del rectángulo. En negrita, lo cargado:

% por celda \ mínimo en el 4 × 4	8	10	12	14	16
60 % (6 de 10)	8	5	5	4	1
70 % (7 de 10)	7	4	4	3	1
80 % (8 de 10)	5	4	**3**	1	1
90 % (9 de 10)	2	1	1	1	1
100 %	1	1	0	0	0

Subir el mínimo de 12 a 14 quita a C y a E (las dos tienen 12 justo) y deja sólo a G. G aguanta en toda la tabla menos en la última fila con mínimo 12 o más. Aflojar a 70 % mete una cuarta. La decisión que de verdad cambia la lista no es el umbral: es si las celdas cuentan como 30 o como 3.

---

5. Lo que no cuadra o no está medido

5.1 El tipo de walk-forward ha cambiado en el YAML y no en lo demás

`assets/_build.yaml:483` dice hoy `wf_type: simulated_is_simulated_oos`. El comentario que tiene justo encima (l.477-482, decisión tuya del 2026-09-24), la skill (`.claude/skills/wfm/SKILL.md:22`) y la corrida real dicen `exact_is_exact_oos`. En el `project.cfx` archivado, el `type="2"` es el del bloque `<WalkForwardMatrix>` de `Retest-Task2.xml`; ojo al buscarlo, porque el primer `<WalkForward type="1">` que aparece en ese fichero es otro bloque (el walk-forward de los cross-checks), no la matriz. `git diff` confirma que la línea 483 es un cambio sin confirmar; no sé de qué sesión viene ni por qué. Una corrida lanzada hoy escribiría el tipo simulado: más rápido, pero no es lo que describen la skill, este documento ni los números de esta corrida. **No medido:** cuánto cambian las celdas entre un modo y otro.

5.2 El score lleva dentro un tramo optimizado

Visto en 3.3. `core/wfmobjectives.py:133-134`; la etiqueta de la ventana (`labels.py:18-19` : «la métrica del walk-forward entero de la celda frente al backtest original») es correcta pero no avisa de que «entero» incluye la primera ventana de optimización. Efecto medido: C6 la cumple el 71 % de las celdas en vez del 34 %; E pasa el área sólo por eso; y la condición favorece a la original de PF bajo. Así es como lo calcula SQX; lo que es decisión tuya es si esa condición debe estar entre las diez.

5.3 El «fuera de muestra» de la WFM y la skill

Visto en 2.9. La skill dice (l.84-86) que las fechas del resultado «tienen que ser las de `oos2`» y que si aparece 2018-2022 «la tarea está releyendo la muestra gastada». La tarea que se escribe de verdad usa `build..oos2` (`wfm.py:127-134`) y **siempre** aparecen 2013-2023: entre el 67 y el 79 % de los días del cosido. La skill describe una versión anterior de la tarea. También son del oro sus cifras «18,7 años, de los que 3,7 son `oos2`» (l.31, l.97; aquí 15,03 y 2,75) y «con 6 tramos, cada pasada dura tres años y medio» (l.97-98; aquí entre 1,4 y 1,8 años).

5.4 El veredicto de la ρ no distingue, y «perversa» sale sola: simulación

Para saber qué veredicto da el estudio a una estrategia **sin ninguna ventaja**, he simulado 300 historias de resultados diarios de puro azar (media cero, 5.490 días), con la misma rejilla de 30 celdas, eligiendo en cada tramo «el mejor» de la ventana de optimización, y les he pasado el mismo cálculo del estudio: ρ por celda, z de Fisher, bootstrap sobre celdas, veredicto.

variante de la simulación	ρ conjunta media	banda 2,5 % – 97,5 %	perversa	ciega	predice
métrica media, sin selección	−0,20	[−0,55, +0,29]	68 %	20 %	12 %
RetDD, sin selección	−0,19	[−0,54, +0,31]	66 %	23 %	11 %
RetDD, el mejor de 20 candidatos parecidos	−0,155	[−0,50, +0,25]	64 %	27 %	10 %

El revisor (Taleb) la rehízo con otra semilla y 400 historias, y la estiró en las direcciones en que yo dudaba:

variante del revisor	perversa	centro de la ρ
días normales	74 %	−0,20 a −0,21
colas gordas (t con 3 grados de libertad)	74 %	ídem
sólo el 12 % de los días con actividad	73 %	ídem
volatilidad agrupada	74 %	ídem
sin solape (la optimización leída sobre una serie independiente)	**36 %** · predice 34 % · ciega 29 %	**0,00**

Entre mi semilla y la suya, la horquilla honesta de «perversa» sin ventaja es **64-74 %**. La forma de la distribución no importa (la ρ es de rangos sobre sumas de bloques largos), así que la reserva «días normales» que tenía este apartado sobra.

Lo que sale de las dos tablas, por orden de importancia:

- **El problema principal es el intervalo, no el sesgo.** La banda del azar es de ±0,4; el intervalo que da el estudio es de ±0,1. Lo que varía la ρ entre historias es de 4,2 a 5,2 veces el error del bootstrap (`measure/correlation.py:56-65`). La última fila lo aísla: **aun corrigiendo el sesgo, el intervalo da un veredicto no ciego el 70 % de las veces** —36 % perversa, 34 % predice— en estrategias sin nada. `zero_band: 0.0` no arregla nada, y mover el centro tampoco: hay que cambiar el intervalo.
- **Además, el centro no es cero: es −0,15 a −0,21.** Es el sesgo por solape de 2.7, y está demostrado: sin solape el centro vuelve a 0,00. Es lo que convierte el 36 % de «perversa» en 64-74 % y deja «predice» en el 8-12 %. La ρ media de las 300 celdas reales es −0,08: menos negativa que la de la simulación sin ventaja.
- **En la corrida real «perversa» salió 5 de 10**, por debajo de lo que da el ruido.

Con la banda de la simulación como vara, las diez ρ reales caen dentro menos la de F (+0,345), que queda por encima del 97,5 %; la de `3.9.284` (−0,42), la más negativa, cae dentro con p ≈ 0,15. Y hay que contar las diez miradas:

```
que alguna de diez salga de la banda por un lado:              1 − 0,975¹⁰ = 22 %
que alguna de diez salga por arriba o por abajo:               1 − 0,95¹⁰  = 40 %
que alguna de diez saque «predice» (sale el 8-12 % sin nada):  1 − 0,90¹⁰  ≈ 65-72 %   (69 %
en la simulación del revisor)
```

La p de una cola del «predice» de F es de 0,007 a 0,02 según la variante; multiplicada por diez miradas queda entre 0,07 y 0,2: no sobrevive. Dicho de otro modo: **ni las cinco «perversas» ni el «predice» de F se distinguen de lo que da el ruido.**

Límites que quedan: 20 candidatos por optimización, no 5.000; sin tendencia del activo; resultados diarios simulados, no operaciones de una estrategia real. La medida de verdad es una población sin ventaja pasada por SQX (4.3, punto 3). Los scripts están en el directorio temporal de la sesión (`scratchpad/wfm/null_rho.py` el mío, `scratchpad/review/check.py` el del revisor), no en el repositorio.

Relacionado: el título del estudio y su glosario («¿lo que optimiza bien predice lo que va bien después?», `many.py:14-30`, README) prometen más de lo que la ρ mide (2.7, último párrafo).

5.5 La deriva divide entre dos columnas de más

Visto en 3.7. `measure/drift.py:30-36` con `inputs/export.py:60-73`; y `drift.py:59` para el «74 parámetros». No cambia ningún veredicto aquí; sí los números que se enseñan (0,63-0,80 en vez de 0,80-0,92).

5.6 `oos2` se cobra con el spread de `oos1`

`sqx/projects/wfm.py:128` toma los costes de un solo tramo (`costs_segment: oos1`) para toda la ventana. La skill lo justifica (l.32-33) diciendo que «hoy `oos1` y `oos2` comparten spread». En NIKKEI225 no: 474,91 frente a 796,12. El tramo `oos2` de la WFM se prueba al 60 % de su spread declarado, y `build` a un 56 % **más** del suyo (304,11). La pestaña «Contra el activo y el azar» usa las operaciones del backtest original que exporta la misma tarea, así que hereda el mismo coste. **No medido:** cuánto baja el resultado de `oos2` con su spread.

5.7 «% OOS» y lo que dice el YAML

`assets/_build.yaml:489` lo describe como «cuánto de cada pasada va fuera de muestra». Es el OOS como porcentaje de la ventana de optimización (2.3): «40 %» es 28,6 % de la pasada.

5.8 SQX «descarta» según el YAML; en realidad marca

`assets/_build.yaml:508-510` dice que SQX descarta la estrategia que no cumple y no la escribe en el databank de salida. La tarea lleva `DeleteFailedStrategies=false` y la skill dice lo contrario que el YAML: marca, no borra. Comprobado en la corrida: 7 marcadas, 10 en el databank.

5.9 Los porcentajes de calibración son de otro activo

Visto en 3.4. Los diez umbrales se eligieron mirando 150 celdas de XAUUSD (5 estrategias del master). En NIKKEI225, C5 pasa del 79 % al 42 % y C8 del 11 % al 26 %. Los umbrales son comunes a todos los activos (`_build.yaml`, bloque `wfm`); **no está medido** en ningún otro.

5.10 El «beneficio de la estrategia» de la tabla del azar no es el de SQX

Para C, la tabla del azar pone 7.021,62 en la fila de la estrategia. Las 115 operaciones de C en `oos2` suman en SQX 4.337,65 (que es además lo que da la tabla año a año: +7,50 − 1,10 − 2,06 = +4,34 % de 100.000). El motor del azar no usa el resultado de SQX: vuelve a poner precio a las operaciones desde las barras (`engines/nulls/simulate.py:15-39`) y compara eso con sus propios traders. La comparación es coherente por dentro —estrategia y traders están medidos con la misma vara—, y el control que da la pestaña («reconcile» 0,997) es una correlación, que no detecta una diferencia de nivel. **La causa de la diferencia (un 62 % más) sigue sin explicar**, y el revisor tampoco la cerró. Candidato no comprobado: el motor mide por su cuenta el valor del punto y el coste por operación a partir de las propias operaciones (`engines/nulls/simulate.py:31-32`), y esas dos medidas pueden no coincidir con las de SQX. Sólo lo he mirado en C. **Hasta cerrarlo, ninguna p de la pestaña del azar es utilizable**: ni la de `10.11.484` (0,031) como aprobado ni las otras nueve como suspenso.

5.11 Detalles de las condiciones

C7 con «mayor que» estricto y C8 con su centinela de 100: 3.2. No son errores; son comportamientos que conviene tener presentes al leer las medianas.

5.12 El README del estudio

Su «79 % de solape» es el caso p = 20 (la media es 70 %); su tabla de primeros veredictos y su «60 de 720 tramos futuros» son de la corrida de XAUUSD de septiembre.

5.13 Lo que no está medido o no he podido comprobar

- **Ninguna tasa de falsos positivos**, ni de la regla del área ni de la ρ. Todo lo que este documento dice del azar es simulación ilustrativa.
- **Dos tramos sin estadísticas de pasada** (2 de 3.300; por eso dos estrategias tienen 329 tramos y la tabla de parámetros 3.598 filas). `core/wftrades.py:67-75` lo anota; no sé por qué faltan.
- **Si las 5.000 pruebas por optimización se hicieron.** Diez estrategias, 3.300 optimizaciones, 1.695 segundos: medio segundo por optimización en modo exacto. No he comprobado en el log de SQX cuántas pruebas hizo cada una.
- **Cómo reparte SQX los 18 escalones dentro del ±35 %.** La tarea sólo le pasa el rango y el máximo de escalones (`wfm.py:67-72`; el `project.cfx` archivado trae `maxSteps="18"`); el «4 %» es la cuenta de la que sale el 18 y no se escribe en ningún sitio que SQX lea.
- **Si algo aguas abajo usa los parámetros recomendados** de la WFM. No he encontrado que nada lo haga.
- **Cuánto cambia todo con el spread de `oos2`** (5.6) y con el modo simulado (5.1).

5.14 Dónde sigue valiendo el informe anterior, y dónde no

`curvas-wfm-2026-10-01` (cinco madres de XAUUSD) sigue valiendo en lo principal, y esta corrida lo confirma con otro activo:

- las celdas no son independientes (allí 1,6 a 3 efectivas; aquí 2,2 a 4,6);
- pasar la marca de SQX no es tener ventaja (allí la única que pasaba era «perversa»; aquí lo es G, la que mejor pasa);
- ninguna bate al activo;
- lo que hay que juzgar es la original en `oos2`, no una celda. De ahí salió la pestaña «Contra el activo y el azar», que ya existe.

Lo que hay que corregir o actualizar:

- **Sus números son del oro**; los de este documento, de NIKKEI225. Ninguno de los dos es «el» valor.
- **Leía «perversa» como un hallazgo.** Con 5.4 delante, es sobre todo un intervalo mal calculado y, encima, un sesgo de la medida. Esto es nuevo y cambia la lectura de aquel informe.
- **Su propuesta B** (bootstrap por bloques de calendario en vez de por celdas, que arreglaría el intervalo de la ρ) **sigue sin construir**; las celdas efectivas siguen sin calcularse en el código.
- No trataba el score (5.2), el coste de `oos2` (5.6) ni la deriva (5.5).

---

6. Cierre

6.1 Las reglas, en una tabla

dónde	regla	estado
`criteria.yaml`, paso 19	—	no existe; «sin regla a propósito»
`tail.py`, paso 19	—	no se juzga; no hay hechos
SQX, celda	cumple ≥ 80 % de 10 condiciones (≥ 8)	activa; marca, no borra
SQX, área	algún 4 × 4 de la matriz 6 × 5 con ≥ 12 celdas	activa; marca, no borra
estudio, ρ	intervalo sobre 0 → predice; contiene 0 → ciega; bajo 0 → perversa	activa; anota
estudio, deriva	mediana de parámetros que cambian ≥ 0,5 → aviso	activa; anota (sale en las diez)
estudio, activo	intervalo de la diferencia de Sharpe sobre / bajo / con el 0	sólo se enseña
estudio, azar	p del Sharpe por operación; pinta a 0,05; mínimo 30 operaciones	sólo se enseña
paso 20	la ρ como pieza: perversa = fallo, y un fallo marca DESCARTAR (`sin_fallo`)	**activa; marca en su `verdict.csv`, el autopiloto no lo aplica; corta sólo con `/curate` a mano**

6.2 El resultado sobre la población real

Diez entran, diez salen. SQX marca siete; el estudio llama perversa a cinco, ciega a cuatro y predice a una. En `oos2`, ninguna se distingue del activo y una de diez baja de 0,05 contra el azar. Las dos lecturas del paso coinciden en dos estrategias (C y E: pasan el área, ciegas) y se contradicen en las dos más claras (F y G).

6.3 Las decisiones que te quedan

Cada una con lo que cuesta en estrategias sobre estas diez.

1. **¿La marca de SQX decide algo?** Hoy no. Como fallo: caen 7 de 10 (quedan C, E, G). Antes de usarla conviene saber que C y E pasan con el mínimo exacto en una sola posición, y que las celdas valen por 2 o 3. Falta: que sea un hecho (4.3).
2. **¿«Perversa» sigue siendo un fallo en el paso 20?** Hoy sí, sobre el papel: marca DESCARTAR a 5 de 10, G incluida, y el autopiloto no lo aplica (sólo `/curate` a mano). Si pasa a «vigilar»: 0. Las simulaciones de 5.4 dicen que ese veredicto sale entre el 64 y el 74 % de las veces sin causa, y que el intervalo en que se apoya es de cuatro a cinco veces demasiado estrecho; es la decisión de esta lista con menos respaldo.
3. **¿El tipo de walk-forward es exacto o simulado?** No cuesta estrategias hoy; cuesta que el YAML, la skill y la corrida digan lo mismo (5.1). Sin medir qué cambia.
4. **¿Con qué spread se prueba `oos2`?** Con el de `oos1` (hoy) o con el suyo, un 68 % mayor. Sin medir; el efecto sólo puede ser a peor en `oos2` y a mejor en `build`.
5. **¿El score se queda entre las diez condiciones?** Tal como es: 3 pasan. Con el OOS solo: 2 (cae E). Quitándolo sin más, las condiciones serían nueve y el 80 % pasaría a exigir 8 de 9: no lo he calculado.
6. **¿Los umbrales de las diez condiciones son comunes a todos los activos?** Hoy sí, calibrados con el oro. C10 no quita nada (97 %) y C5 quita el doble que allí. Recalibrar por activo cambia la lista; cuánto, sin medir.
7. **¿Qué tramo cuenta?** La matriz suma `build`, `oos1` y `oos2`. Si lo que quieres saber es qué pasa en dato limpio, eso son 2 a 4 tramos por celda y hoy no hay ningún número del paso que lo aísle salvo la pestaña del azar.
8. **¿Algo de la pestaña «Contra el activo y el azar» pasa a ser regla?** p ≤ 0,05: caen 9 de 10. Batir al activo: caen las 10. Con 2,75 años y un centenar de operaciones, ninguna de las dos puede separar una estrategia buena de una normal (el intervalo es de ±1,2 de Sharpe). Y antes de todo hay que cerrar 5.10: mientras tanto ninguna p de esa pestaña es utilizable.
9. **¿El paso 19 debe guardar hechos?** Es condición previa de las ocho anteriores. No cuesta estrategias.

6.4 El límite que no cambia

Tres, y ninguno se arregla moviendo un umbral:

- **`oos2` son 2,75 años.** Es el único tramo que nadie ha usado para elegir, y con él un Sharpe se conoce con un margen de ±1,2. La WFM no añade dato limpio: reparte el mismo `oos2` en 30 troceos. Entre el 67 y el 79 % de lo que llama OOS es `build` y `oos1`, y de ahí salen tres cosas: C1, C2 y C7 están infladas por la selección del paso 8 en `oos1`; la única evidencia limpia del paso, la pestaña del azar, no necesita la WFM; y el paso gasta miradas a `oos2` en un veredicto que sin ventaja sale «perversa» el 64-74 % de las veces.

- **Treinta celdas de una historia son un puñado de pruebas** (entre 1,6 y 4,6 según la definición; el número exacto no existe). Toda regla que cuente celdas hereda eso, se ponga el mínimo en 12 o en 14.
- **Sin una población sin ventaja pasada por la cadena, no se sabe qué deja pasar ninguna de estas reglas.** Es lo que dice `criteria.yaml:59`, y este paso es de los que más lo necesitan: sus dos lecturas —el área y la ρ— dan, en simulación, un resultado «con significado» en una parte grande de los casos en que no hay nada.